

VPN-LAN Protocol Reference — Manual, Tutorials & FAQ

Revision: 1 **Last modified:** 2026-07-01T16:30:58Z **Status:** Active — per-protocol reference manual, step-by-step tutorials, and FAQ for the VPN-LAN service-access feature (Phase 9 docs of [../design/vpn_lan_access/PLAN.md](#) §5) **Authority:** Inherits constitution/Constitution.md per §11.4.35. Companion to the operator setup guide [vpn_lan_bridge_setup.md](#) (which owns the .env setup + doctor verdicts — this document does NOT repeat them), the architecture diagrams [../design/vpn_lan_access/architecture.md](#), and the design source-of-truth [../design/vpn_lan_access/PLAN.md](#). **Feature workstream:** feature/vpn-aware-dynamic-routing (§11.4.167)

0. How to use this reference

This is the **per-protocol manual**. For each protocol family it gives, in a fixed shape:

1. **What works over this VPN and why** — the primitive (route / Squid-proxy / reflect / structurally-impossible) and the deciding fact.
2. **The exact test that proves it** — the tests/vpn_lan/*.sh script and what its PASS requires (real captured evidence, never a metadata-only PASS — §11.4.69).
3. **The env vars it reads** — the decoupled contract (§11.4.28); real values live in a gitignored .env, never in the tree (§11.4.10).
4. **A step-by-step tutorial** — "how to use it once the bridge is up".
5. **FAQ** — 2-3 entries per protocol.

Before any of this works you must bring the bridge up. That procedure — copying .env.example to .env, filling the 6-var contract, and reading the sword_doctor.sh verdict — lives in the setup guide [vpn_lan_bridge_setup.md](#) and is **not** repeated here. The one-line prerequisite for every tutorial below:

```

set -a; . ./env; set +a          # load the gitignored bridge
contract                         # must print: BRIDGE: UP (exit
scripts/sword_doctor.sh         0)

```

If the doctor prints `BRIDGE: SKIP:network_unreachable_external` the bridge is **down** and every protocol test below will honestly SKIP (exit 2) — that is by design, not a failure (see the top-level FAQ §11 and the setup guide §5).

Anti-bluff contract for every protocol below (FACT — read the test headers): each `tests/vpn_lan/*.sh` sources `tests/lib/sword_bridge.sh`, calls `bridge_require` **first**, and when the bridge is down prints `SKIP:network_unreachable_external` and exits 0. A PASS is emitted **only** through `ab_pass_with_evidence`, which refuses to PASS unless the cited evidence file exists and is non-empty. An absent `share/tool/device` **SKIPS**, it never **PASSES**; a reachable-but-broken service **FAILs**. There is no metadata-only, config-only, or absence-of-error PASS anywhere in this feature (§11.4.69).

1. SMB / CIFS / NMB (NetBIOS) — file shares

What works & why. SMB/CIFS is a **unicast IP** file service — it **routes** over the L3 VPN. A mount needs L3 routing, **not** SOCKS5/Squid (that is the wrong primitive for mounts). NetBIOS name resolution (NMB, 137-138) uses broadcast/multicast which is **not routed across L3**, so you target the share by its **10.x IP** in the UNC (e.g. `//10.6.100.221/share`), not by NetBIOS name — the unicast fallback. Ports: SMB/CIFS **445**, NMB **137-138** (FACT, [PLAN.md §2](#)).

The test that proves it. `tests/vpn_lan/smb_nfs_roundtrip.sh` (Phase 2). It does a **write** → **read-back** → **sha256** round-trip via `smbclient` (preferred, no root) and **PASSES only** when `src_sha256 == readback_sha256` (integrity: MATCH) — real bytes survived the trip (§11.4.5). Evidence: `qa-results/vpn_lan/phase2/<UTC-ts>/smb/roundtrip.evidence`.

Env vars it reads (all optional; unset ⇒ `SKIP:feature_disabled_by_config`):

Env var	Meaning
HELIX_VPN_SMB_UNC	UNC of the VPN share, e.g. <code>//10.6.100.221/share</code>
HELIX_VPN_SMB_USER	username (omit for anonymous / -N)

Env var	Meaning
HELIX_VPN_SMB_PASS	password — never logged / never on argv (§11.4.10)
HELIX_VPN_SMB_DOMAIN	optional workgroup / domain

Tutorial — use an SMB share once the bridge is up:

1. Confirm the bridge: `scripts/sword_doctor.sh` prints `BRIDGE: UP`.
2. Point at the share by IP in your `.env`: `HELIX_VPN_SMB_UNC=//10.6.100.221/share` (plus `HELIX_VPN_SMB_USER / HELIX_VPN_SMB_PASS` if not anonymous).
3. Interactively list it: `smbclient //10.6.100.221/share -U "$HELIX_VPN_SMB_USER" -c 'ls'`.
4. Or run the round-trip proof: `set -a; . ./env; set +a; tests/vpn_lan/smb_nfs_roundtrip.sh` and read the `PASS: SMB/CIFS ... [evidence: .../smb/roundtrip.evidence]` line — open that evidence file to see the matching sha256.

FAQ.

- *Why target by IP, not by hostname?* Because NetBIOS name resolution is broadcast/multicast and is not routed across the L3 VPN; the unicast `10.x` IP in the UNC is the routable path.
- *Do I need root?* No — the test prefers `smbclient` (userspace, no mount). `mount.cifs` would need root and is not exercised by the default test.
- *The test says `SKIP:network_unreachable_external` even though the bridge is up — why?* The bridge is up but `smbclient` could not reach/authenticate the specific share; that is a topology/config gap (wrong UNC, credentials, or share offline), reported honestly as `SKIP` rather than a false `PASS`.

2. NFS — file shares

What works & why. NFS is a **unicast IP** mount (2049 + auxiliary ports) — it **routes** over the L3 VPN, same as SMB. Not a proxied service (§11.4.6, [PLAN.md](#) §2).

The test that proves it. `tests/vpn_lan/smb_nfs_roundtrip.sh` (Phase 2, NFS half). Same **write → read-back → sha256** integrity loop as SMB, via `cp` into a mounted export, `PASSes` only on integrity: `MATCH`. Evidence: `qa-results/vpn_lan/phase2/<UTC-ts>/nfs/roundtrip.evidence`.

Env vars it reads:

Env var	Meaning
HELIX_VPN_NFS_MOUNTED	

Env var	Meaning
	path to an already-mounted NFS export (no root needed — preferred)
HELIX_VPN_NFS_EXPORT	server:/export to temp-mount into a scratch dir (needs root ; unmounted + removed on every exit path, §11.4.14)

Tutorial:

1. Bridge up (BRIDGE: UP).
2. Either mount the export yourself and set
HELIX_VPN_NFS_MOUNTED=/mnt/vpn_export, **or** set
HELIX_VPN_NFS_EXPORT=10.6.100.221:/export (root path — the test temp-mounts it).
3. Run set -a; . ./env; set +a; tests/vpn_lan/smb_nfs_roundtrip.sh and read the PASS: NFS ... [evidence: .../nfs/roundtrip.evidence] line.

FAQ.

- *Which of the two NFS vars should I use?* Prefer HELIX_VPN_NFS_MOUNTED — it needs no root and the test just reads/writes inside a directory you already control.
- *Does the temp-mount leave anything behind?* No — the temp mount is unmounted and the temp dir removed on every exit path via the cleanup trap (§11.4.14).

3. FTP / FTPS / SFTP / WebDAV — file transfer

What works & why.

- **FTP control (21)** routes as plain unicast TCP; the **passive data channel** uses the server's pinned passive-port range, which you **route** too. **FTPS** (explicit AUTH TLS or implicit 990) encrypts the control channel, so NAT/proxy ALGs cannot rewrite the PASV reply — **routing** (not proxying) is the clean fit (FACT, [PLAN.md](#) §5 Phase 3).
- **SFTP (22)** is a single connection over SSH — routes/tunnels trivially; the recommended modern path.
- **WebDAV** is **HTTP** (RFC 4918) — it goes through the **existing Squid** (PROPFIND returns 207 Multi-Status), **no new component**. INFERENCE-free: WebDAV-over-HTTP through a forward proxy is standard; the exact Squid method-allowlist is a deployment detail (see FAQ).

The test that proves it. tests/vpn_lan/ftp_sftp_webdav.sh (Phase 3):

- FTP (T3.1): passive directory-list + fetch via curl.
- SFTP (T3.2): **byte round-trip** write → read-back → sha256 via sftp — PASS only on match.
- WebDAV (T3.3): PROPFIND through Squid — PASS on a captured 207 body. Evidence: qa-results/vpn_lan/phase3/<UTC-ts>/{ftp,sftp,webdav}/.

Env vars it reads:

Env var	Meaning
HELIX_VPN_FTP_URL	base FTP URL, e.g. ftp://10.6.100.221/pub/
HELIX_VPN_FTP_USER / HELIX_VPN_FTP_PASS	FTP credentials (omit user for anonymous; pass never logged)
HELIX_VPN_SFTP_HOST	SFTP host (10.x)
HELIX_VPN_SFTP_USER	SFTP user
HELIX_VPN_SFTP_PORT	SFTP port (default 22)
HELIX_VPN_SFTP_DIR	remote dir (default .)
HELIX_VPN_SFTP_KEY	identity file (key-based, BatchMode — no interactive password)
HELIX_VPN_WEBDAV_URL	WebDAV collection URL, e.g. http://10.6.100.221/dav/
HELIX_SQUID_PROXY	proxy endpoint (default http://127.0.0.1:34128)

Tutorial — SFTP round-trip (the recommended path):

1. Bridge up. Set HELIX_VPN_SFTP_HOST=10.6.100.221, HELIX_VPN_SFTP_USER=<user>, and a key: HELIX_VPN_SFTP_KEY=~/.ssh/id_vpn (key-based, no interactive password).
2. Interactively: sftp -i ~/.ssh/id_vpn <user>@10.6.100.221.
3. Proof: set -a; . ./env; set +a; tests/vpn_lan/ftp_sftp_webdav.sh; read the PASS: SFTP ... line + the sha256-match evidence.

Tutorial — WebDAV through the existing Squid:

1. Bridge up + base proxy running. Set HELIX_VPN_WEBDAV_URL=http://10.6.100.221/dav/.
2. Interactively: curl -x "\$HELIX_SQUID_PROXY" -X PROPFIND -H 'Depth: 1' "\$HELIX_VPN_WEBDAV_URL" — expect a 207 Multi-Status XML body.
3. Proof: the Phase-3 test captures the 207 body as evidence.

FAQ.

- *Why does WebDAV need no new service?* It is HTTP-shaped, so it rides the **existing Squid** — the same proxy that already carries HTTP. No SOCKS hop, no new daemon.
 - *My Squid rejects PROPFIND / MKCOL — why?* Older Squid builds gate uncommon HTTP methods; enable `extension_methods` and ensure the WebDAV origin's TLS port is in Squid's `SSL_Ports` for `CONNECT` (INFERENCE on your specific Squid version — [PLAN.md §5 T3.3](#)).
 - *FTP vs SFTP — which should I use?* SFTP: one connection, routes trivially, encrypted by default. FTP passive works but needs the pinned passive range routed and is ALG-fragile under FTPS. SFTP is the recommended modern path.
-

4. IMAP / SMTP-submission / POP3 — email

What works & why. All three mail protocols are **unicast TCP** — they **route** over the L3 VPN. Implicit-TLS ports (IMAPS **993**, POP3S **995**, submission **465**) are **preferred** over plaintext-upgradable STARTTLS ports (RFC 8314) where the choice exists (FACT, [PLAN.md §4.4](#)). **Critical security rule:** route **authenticated submission (587/465)** to VPN clients, but **never** expose an anonymous `CONNECT-to-:25` — `helix_proxy` must not become an open relay / spam conduit (§4.3).

The test that proves it. `tests/vpn_lan/email_roundtrip.sh` (Phase 4) — four checks:

- **T4.1 IMAPS (993):** `LOGIN + LIST` returns a real mailbox listing.
- **T4.2 SMTP submission (465 implicit / 587 STARTTLS):** authenticated send is **250-accepted**.
- **T4.3 POP3S (995):** retrieves the just-sent round-trip token.
- **T4.4 OPEN-RELAY NEGATIVE TEST (MANDATORY, §4.3):** an **unauthenticated** relay to an **external** domain **MUST be refused** — a captured `4xx/5xx` refusal is the PASS; a captured `2xx` acceptance of the external `RCPT` is a **FAIL** (`helix_proxy` accepted an open relay). Evidence: `qa-results/vpn_lan/phase4/<UTC-ts>/` (+ a `MANIFEST.md` with config but **no** credentials).

Env vars it reads:

Env var	Meaning / default
<code>HELIX_MAIL_HOST</code>	mail server host (default <code>HELIX_BRIDGE_HOST</code>)
<code>HELIX_MAIL_IMAPS_PORT</code>	default 993 (implicit TLS, RFC 8314)

Env var	Meaning / default
HELIX_MAIL_POP3S_PORT	default 995 (implicit TLS)
HELIX_MAIL_SUBMISSION_PORT	default 465 (implicit TLS)
HELIX_MAIL_SUBMISSION_TLS	implicit starttls (default implicit)
HELIX_MAIL_RELAY_PROBE_PORT	default = submission port
HELIX_MAIL_RELAY_PROBE_TLS	implicit starttls (default = submission TLS)
HELIX_MAIL_EXTERNAL_RELAY_DOMAIN	external domain for the §4.3 negative test (default example.com)
HELIX_MAIL_USER / HELIX_MAIL_PASS	account credentials — never logged / never on argv (§11.4.10)
HELIX_MAIL_FROM / HELIX_MAIL_TO	envelope from/to (default = HELIX_MAIL_USER)
HELIX_MAIL_TIMEOUT	per-dialog timeout secs (default 20)
HELIX_MAIL_PROBE_TIMEOUT	TCP-connect probe timeout secs (default 6)
EMAIL_ROUNDTRIP_EVIDENCE_DIR	override the evidence dir

Tutorial — prove the mail round-trip + open-relay guard:

1. Bridge up. Set HELIX_MAIL_HOST, HELIX_MAIL_USER, HELIX_MAIL_PASS in .env (implicit TLS ports 993/995/465 are the defaults).
2. Run `set -a; . ./env; set +a; tests/vpn_lan/email_roundtrip.sh`.
3. Read the four verdict lines: IMAPS LIST PASS, submission 250 PASS, POP3S retrieve PASS, and the **open-relay negative** PASS (which means the unauthenticated external relay was correctly **refused**).

FAQ.

- *Why is a refused relay a PASS?* The T4.4 negative test proves helix_proxy is **not** an open relay. A refusal (4xx/5xx) is the desired secure behaviour; an acceptance (2xx) of an unauthenticated external RCPT would be a FAIL — a spam-conduit defect.
- *Why implicit TLS instead of STARTTLS?* Implicit-TLS ports (993/995/465, RFC 8314) are not vulnerable to STARTTLS-stripping downgrade; prefer them where the server offers both.
- *Where do my mail credentials go?* Only in the gitignored .env; they are passed to the TLS dialog via an in-process printf into

stdin — never on an external process argv, never logged (§11.4.10).

5. Chromecast / DIAL — casting

What works & why. Cast splits in two: **discovery** is multicast (`_googlecast._tcp` mDNS

- DIAL SSDP M-SEARCH) and needs the **remote-side reflector** (Phase 5, §7 below); **control** is **unicast TCP** and **routes** over the VPN — `eureka_info` HTTP on **8008** and CASTV2 on **8009** (TLS). Once the device's 10.x IP is known, control routes directly (FACT, [PLAN.md](#) §5 Phase 6). This is the **routeable replacement for Miracast** (§8).

The test that proves it. `tests/vpn_lan/chromecast_dial.sh` (Phase 6):

- **T6.2 Control:** GET `http://<ip>:8008/setup/eureka_info` over the routed path — the real JSON name field is the device-identity evidence.
- **T6.3 Liveness (§11.4.107):** a CASTV2 status **transition** observed across two reads (advancing state, **not** a single frozen frame) — a real transition is required for the liveness PASS.
- **T6.1 Discovery:** depends on the Phase-5 reflector; direct-IP control is used when a device IP is configured. Evidence: `qa-results/vpn_lan/phase6/<UTC-ts>/{discovery,eureka,castv2}/`.

Env vars it reads:

Env var	Meaning / default
HELIX_VPN_CAST_IP	cast device 10.x address (no device = SKIP)
HELIX_VPN_CAST_EUREKA_PORT	eureka_info HTTP port (default 8008)
HELIX_VPN_CAST_CASTV2_PORT	CASTV2 TLS control port (default 8009)
HELIX_VPN_CAST_STATUS_CMD	operator-supplied cast-status command (e.g. a <code>go-chromecast/catt</code> wrapper), run twice to observe a transition
HELIX_VPN_CAST_REFLECTOR	Phase-5 reflector marker; when set + <code>avahi-browse</code> present, discovery is attempted

Tutorial:

1. Bridge up + (for discovery) the Phase-5 reflector deployed. Set `HELIX_VPN_CAST_IP` to the device's 10.x address.
2. Interactively read identity: `curl http://<cast-ip>:8008/setup/eureka_info` — look for the "name" field.
3. Proof: `set -a; . ./env; set +a; tests/vpn_lan/chromecast_dial.sh`; read the eureka name PASS + the CASTV2 status-transition PASS.

FAQ.

- *Do I need the reflector to control a Cast device?* No — **control** routes directly once you know the device's 10.x IP. You only need the reflector for **discovery** (finding the device without knowing its IP).
 - *Why two status reads?* A single frame can be a frozen/stale state; a real **transition** between two reads proves the device is genuinely live (§11.4.107).
 - *This is read-only?* Yes — the test does a read-only `eureka_info` GET and read-only status reads. No media is launched, no device state is changed.
-

6. ADB — Android device bridge (access / debug / connect / flash)

What works & why. ADB over TCP (5555) is **unicast** — `adb connect 10.x:5555` **routes** over the VPN with **no proxy hop** (central adb-server model: one adb server, multiple remote devices). **Flash is the honest boundary:** fastboot is a **USB-level** protocol, not a routable IP service — the routable path is **usbip** (USB-over-IP) from a remote host with the device physically attached. Network fastboot is honestly USB-bound (FACT, [PLAN.md](#) §5 Phase 7 / recon 4). Any real-device flash is **operator-gated** for target-hardware safety (§11.4.133 / §11.4.122).

The test that proves it. `tests/vpn_lan/adb_over_vpn.sh` (Phase 7):

- **T7.1 Connect:** `adb connect <host>:5555` over the routed path.
- **T7.3 Debug:** `adb -s <serial> shell getprop ro.product.model` returns real device-model content (the captured evidence).
- **T7.4 Flash:** documented as the honest USB-bound boundary; the script **never flashes** — it SKIPS the flash sub-check `operator_attended`.
- **SAFETY (§11.4.174):** touches **only** the env-configured serial — never `adb kill-server`, never a blanket `adb disconnect`, never any other serial in adb devices (operator/lab devices are off-limits). `adb disconnect <our-serial>` runs in the cleanup trap

(§11.4.14). Evidence: qa-results/vpn_lan/phase7/<UTC-ts>/
{connect,debug,flash}/.

Env vars it reads:

Env var	Meaning / default
HELIX_VPN_ADB_HOST	device 10.x address (no host ⇒ SKIP)
HELIX_VPN_ADB_PORT	adb TCP port (default 5555)

Tutorial:

1. Bridge up. Set HELIX_VPN_ADB_HOST=10.6.100.50 (your device's 10.x address).
2. Interactively: adb connect 10.6.100.50:5555 then adb -s 10.6.100.50:5555 shell getprop ro.product.model.
3. Proof: set -a; . ./env; set +a; tests/vpn_lan/adb_over_vpn.sh; read the getprop content PASS. Disconnect happens automatically in cleanup.

FAQ.

- *Can I flash a device over the VPN?* Not with network fastboot — it is USB-bound. Use usbip (USB-over-IP) from a host with the device attached; any real flash is operator-gated (§11.4.133).
 - *Will the test disturb my lab devices?* No — it acts **only** on the serial in HELIX_VPN_ADB_HOST and never runs kill-server or a blanket disconnect (§11.4.174).
 - *Do I need a SOCKS/HTTP proxy for adb?* No — 5555 routes directly over the L3 VPN; no proxy hop.
-

7. mDNS / SSDP / WS-Discovery / DNS-SD — discovery reflector

What works & why. Service **discovery** is **multicast** and **does not cross the L3 VPN** — routers do not forward the discovery groups across a subnet boundary. mDNS/DNS-SD use the link-local group 224.0.0.251:5353 (RFC 6762/6763; TTL-1, routers MUST NOT forward); SSDP and WS-Discovery use the administratively-scoped group 239.255.255.250:1900 and :3702 (RFC 2365; no multicast-routing state across the tunnel). The fix is a **remote-side reflector** on the 10/8 subnet — **Avahi enable-reflector=yes** for mDNS + an **SSDP relay** for 1900/3702 that rewrites LOCATION to a routable 10.x address (FACT for the mechanism; the exact SSDP-relay binary is **INFERENCE** — a

deployment choice, [reflector_design.md](#) §3.2). The reflector solves **discovery only** — once a device is enumerated, control routes as unicast (§5/§6).

The test that proves it. `tests/vpn_lan/discovery_reflect.sh` (Phase 5):

- **T5.2 (SCORED):** `avahi-browse -rpt <svc-type>` surfaces a **real** remote service resolved (^=) through the reflector — the non-empty resolved browse output is the enumeration evidence.
- **SSDP M-SEARCH** to `239.255.255.250:1900` is **supplementary** (non-scored) context, never drives PASS/FAIL.
- No reflector configured $\Rightarrow$ `SKIP:feature_disabled_by_config`; no `avahi-browse` $\Rightarrow$ `SKIP:topology_unsupported`. Evidence: `qa-results/vpn_lan/phase5/<UTC-ts>/{mdns,ssdp}/`.

Env vars it reads:

Env var	Meaning / default
HELIX_VPN_REFLECTOR	Phase-5 reflector marker (host/addr). Present $\Rightarrow$ enumeration attempted; unset $\Rightarrow$ <code>SKIP:feature_disabled_by_config</code>
HELIX_VPN_REFLECT_SVCTYPE	DNS-SD service type to browse (default <code>_services._dns-sd._udp</code> , the meta-query; e.g. <code>_googlecast._tcp</code> for Cast-only)
HELIX_VPN_SSDP_ADDR	SSDP multicast addr (default <code>239.255.255.250</code>)
HELIX_VPN_SSDP_PORT	SSDP port (default <code>1900</code>)

Tutorial:

1. Bridge up **and** a reflector deployed on the remote subnet (operator-gated — see FAQ). Set `HELIX_VPN_REFLECTOR=<reflector-host>`.
2. Interactively enumerate: `avahi-browse -rpt _services._dns-sd._udp` — resolved (=) lines are remote services surfaced through the reflector.
3. Proof: `set -a; . ./env; set +a; tests/vpn_lan/discovery_reflect.sh`; a PASS requires a real service enumerated through the reflector.

FAQ.

- *Why can't I just `avahi-browse` the remote subnet directly?*
Because multicast discovery is subnet-local by standard mandate (link-local / administratively-scoped groups); an L3 router (the VPN) drops it. You must reflect it onto the subnet where your client lives.

- *Why is deploying the reflector operator-gated?* Deploying it **changes a remote host** (starts a daemon, joins multicast groups, re-emits traffic). Under §11.4.122 `helix_proxy` asks the operator first (§11.4.66 interactive options) and only deploys on explicit approval.
 - *What tool is the SSDP relay?* The **requirement** (an SSDP-aware relay that rewrites LOCATION to a routable 10.x) is FACT; the exact binary is **INFERENCE** — pinned + cited at deployment time (§11.4.150), never asserted here.
-

8. Miracast — the structurally-impossible boundary

What works & why — it does NOT, and here is the honest reason. Miracast rides **Wi-Fi Direct**, a **Layer-2 radio P2P group** negotiated directly between a source and a sink in RF proximity — **there is no routable IP hop** for an L3 VPN to carry. To "route Miracast over the VPN" the VPN would have to manufacture an 802.11 radio association between two devices that are not in RF proximity, which nothing at Layer 3 can do. Classified **Won't-fix: structurally-impossible** (§11.4.112) with cited Wi-Fi Alliance / Wi-Fi Direct evidence — see [../design/vpn_lan_access/miracast_verdict.md](#).

The "test". There is **no** Miracast-over-VPN test and there never will be — authoring one that appears to pass would be a §11.4 / §11.4.107 bluff (there is nothing real to route). The cited spec text **is** the artifact for this verdict.

The routable alternative: Google Cast / DIAL (§5) — its control plane routes over the VPN as ordinary unicast TCP; only its discovery needs the reflector (§7). If you truly need Miracast, use it **locally at the remote site** (a sink co-located with the source forms its Wi-Fi-Direct group in RF proximity) and drive the setup via whatever IP-based control the VPN *can* reach.

FAQ.

- *Is Miracast just "not implemented yet"?* No — it is **structurally impossible** over an L3 VPN by the standard's design (Wi-Fi-Direct L2), not a missing feature. Reopen requires **new** evidence the platform constraint changed (§11.4.34 / §11.4.7), never re-deriving the same impossibility.
 - *What should I use instead?* Google Cast / DIAL (§5) — the routable "cast to a remote display" capability.
-

9. Quick-reference: protocol → primitive → test → key env var

Protocol	Primitive	Test script	Key target env var
SMB/CIFS/ NMB	L3-route	smb_nfs_roundtrip.sh	HELIX_VPN_SMB_UNC
NFS	L3-route	smb_nfs_roundtrip.sh	HELIX_VPN_NFS_MOUNTED / HELIX_VPN_NFS_EXPORT
FTP/FTPS	L3-route (+ pinned PASV)	ftp_sftp_webdav.sh	HELIX_VPN_FTP_URL
SFTP	L3-route	ftp_sftp_webdav.sh	HELIX_VPN_SFTP_HOST
WebDAV	Squid-proxy (existing)	ftp_sftp_webdav.sh	HELIX_VPN_WEBDAV_URL + HELIX_SQUID_PROXY
IMAP/ IMAPS	L3-route	email_roundtrip.sh	HELIX_MAIL_HOST (+ 993)
SMTP submission	L3-route (auth only)	email_roundtrip.sh	HELIX_MAIL_SUBMISSION_PORT (465/587)
POP3/ POP3S	L3-route	email_roundtrip.sh	HELIX_MAIL_POP3S_PORT (995)
Chromecast/ DIAL control	L3-route	chromecast_dial.sh	HELIX_VPN_CAST_IP
ADB (connect/ debug)	L3-route	adb_over_vpn.sh	HELIX_VPN_ADB_HOST
ADB flash	usbip (USB- over-IP)	adb_over_vpn.sh (SKIPs flash)	— (operator-gated)
mDNS/ SSDP/WS-D/ DNS-SD	remote- reflector	discovery_reflect.sh	HELIX_VPN_REFLECTOR
Miracast	structurally- impossible	— (no test)	—

10. Glossary

- **L3-route** — carry the protocol as ordinary unicast IP packets over the ppp0 VPN link.
- **Squid-proxy** — carry an HTTP-shaped protocol (WebDAV) through the **existing** Squid; no new component.
- **Remote-reflector** — a daemon on the 10/8 subnet that re-emits multicast discovery so a helix_proxy-side client can

enumerate remote services (Avahi enable-reflector + SSDP relay).

- **structurally-impossible** — forbidden by the protocol's own design (Miracast = Wi-Fi-Direct L2), not merely unimplemented (§11.4.112).
 - **honest SKIP** — a test that reports SKIP:<closed-set-reason> (e.g. network_unreachable_external) because a precondition is genuinely absent — **not** a failure and **not** a fake PASS (§11.4.3 / §11.4.69).
 - **usbip** — USB-over-IP: carries the USB-level fastboot protocol from a remote host with the device physically attached (the only routable path for flashing).
-

11. Top-level FAQ — the honest boundaries

Q1. Why do the tests SKIP instead of PASS when I have not connected the VPN? Because the live VPN connection is **operator-supplied** (secrets + credentials live outside the tree). The autonomous slate must never manufacture a green result for a path it cannot actually exercise, so every test calls `bridge_require` first and, when the bridge is down, prints `SKIP:network_unreachable_external` and exits 2. A SKIP means "the bridge is down and the harness told you the truth" — it is the anti-bluff design working, not a failure (§11.4.3 / §11.4.69).

Q2. Why can't Miracast work over the VPN? Miracast rides **Wi-Fi Direct**, a Layer-2 radio peer-to-peer group between two devices in RF proximity — there is **no routable IP hop** for an L3 VPN to carry. It is a structural impossibility of the standard, not a missing feature (§11.4.112). Use **Google Cast / DIAL** instead (§5), whose control plane routes over the VPN. Full verdict: [../design/vpn_lan_access/miracast_verdict.md](design/vpn_lan_access/miracast_verdict.md).

Q3. Why does service discovery need a reflector on the remote side? Multicast discovery (mDNS 224.0.0.251:5353, SSDP/WS-Discovery 239.255.255.250:1900/3702) is **subnet-local by standard mandate** — link-local and administratively-scoped groups are dropped by routers (an L3 VPN is a router). To enumerate remote services you deploy a **remote-side reflector** (Avahi enable-reflector + SSDP relay). That deployment changes a remote host, so it is **operator-gated** (§11.4.122). Full argument: [../design/vpn_lan_access/reflector_design.md](design/vpn_lan_access/reflector_design.md).

Q4. Why do mounts (SMB / NFS) route instead of going through SOCKS5/Squid? Because they are **unicast IP services that route** over the L3 VPN — SOCKS5/Squid is the wrong primitive for mounts and discovery. The VPN is L3-routed, so the

correct primitive is a routed gateway plus a **scoped SSRF allowlist**, not a SOCKS5 hop ([../design/vpn_lan_access/PLAN.md §2](#)). Only HTTP-shaped services (WebDAV) use the existing Squid.

Q5. Will widening egress to the VPN subnet weaken the SSRF hardening? No. The RFC1918 / link-local / loopback / metadata **block stays as the floor**, and only a **narrow carve-out** for HELIX_BRIDGE_SUBNET is added **above** the internal-deny in Dante's first-match order. Every other private range stays denied, and a paired mutation proves the allowlist has teeth (an out-of-allowlist target still denies). The email open-relay guard is a second boundary: authenticated submission routes, anonymous CONNECT-to:25 is never exposed (architecture diagrams §4: [../design/vpn_lan_access/architecture.md](#)).

Q6. Can I flash a device over the VPN? Not with network fastboot — it is USB-level, not a routable IP service. It is carried via **usbip** (USB-over-IP) from a remote host with the device attached. Any real-device flash is **operator-gated** for target-hardware safety (§11.4.133 / §11.4.122).

Q7. Where do secrets live? Do any go in .env.example? Never in .env.example (names and illustrative paths only). Real values go in the gitignored .env (never committed); live-connection credentials live **outside the tree** entirely (§11.4.10 / §11.4.30). Passwords are never logged and never placed on a process argv.

Q8. How do I run all the protocol tests at once? Source your .env and run each tests/vpn_lan/*.sh script; each self-gates on the bridge and reports one verdict per protocol. With the bridge **down** they all honestly SKIP and exit 0; with the bridge **up** and targets configured they produce real round-trip evidence under qa-results/vpn_lan/phase*/<UTC-ts>/.

Sources

- [../design/vpn_lan_access/PLAN.md](#) — §2 routing map, §3 bridge contract, §4 security (open-relay guard, SSRF), §5 per-phase protocol coverage.
- [../design/vpn_lan_access/architecture.md](#) — topology, per-protocol routing decision table, data-flow + SSRF trust-boundary diagrams.
- [../design/vpn_lan_access/reflector_design.md](#) — Phase-5 multicast reflector (RFC 6762/6763/5771/2365, Avahi enable-reflector).
- [../design/vpn_lan_access/miracast_verdict.md](#) — §11.4.112 structural-impossibility verdict (Wi-Fi Alliance / Wi-Fi Direct).
- [vpn_lan_bridge_setup.md](#) — the operator setup guide (the .env contract, sword_doctor.sh verdicts, honest-SKIP behaviour) — the prerequisite for every tutorial here.

- Test scripts (the per-protocol FACTs): tests/vpn_lan/
smb_nfs_roundtrip.sh, tests/vpn_lan/ftp_sftp_webdav.sh, tests/
vpn_lan/email_roundtrip.sh, tests/vpn_lan/chromecast_dial.sh,
tests/vpn_lan/adb_over_vpn.sh, tests/vpn_lan/
discovery_reflect.sh, tests/lib/sword_bridge.sh.